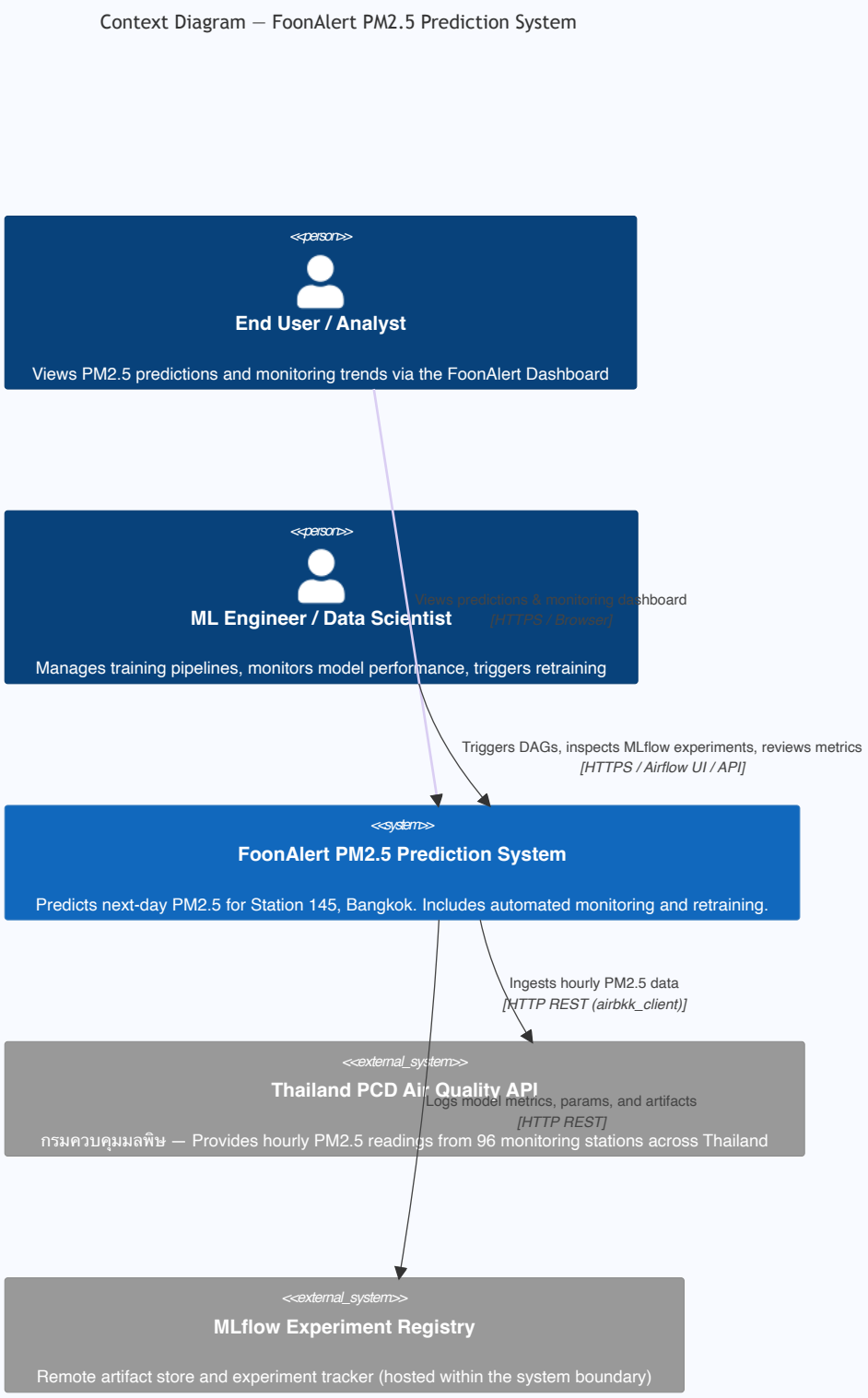
Part 1: System Architecture Overview — C4 Model

System: FoonAlert PM2.5 Prediction System
Deployment: `43.209.207.187` (AWS EC2)
Goal: Predict next-day (24-hour) PM2.5 concentration (μg/m³) for Station 145 — Bangkhuntien, Bangkok

---

Level 1 — Context Diagram

> *System scope, users, and external systems.*



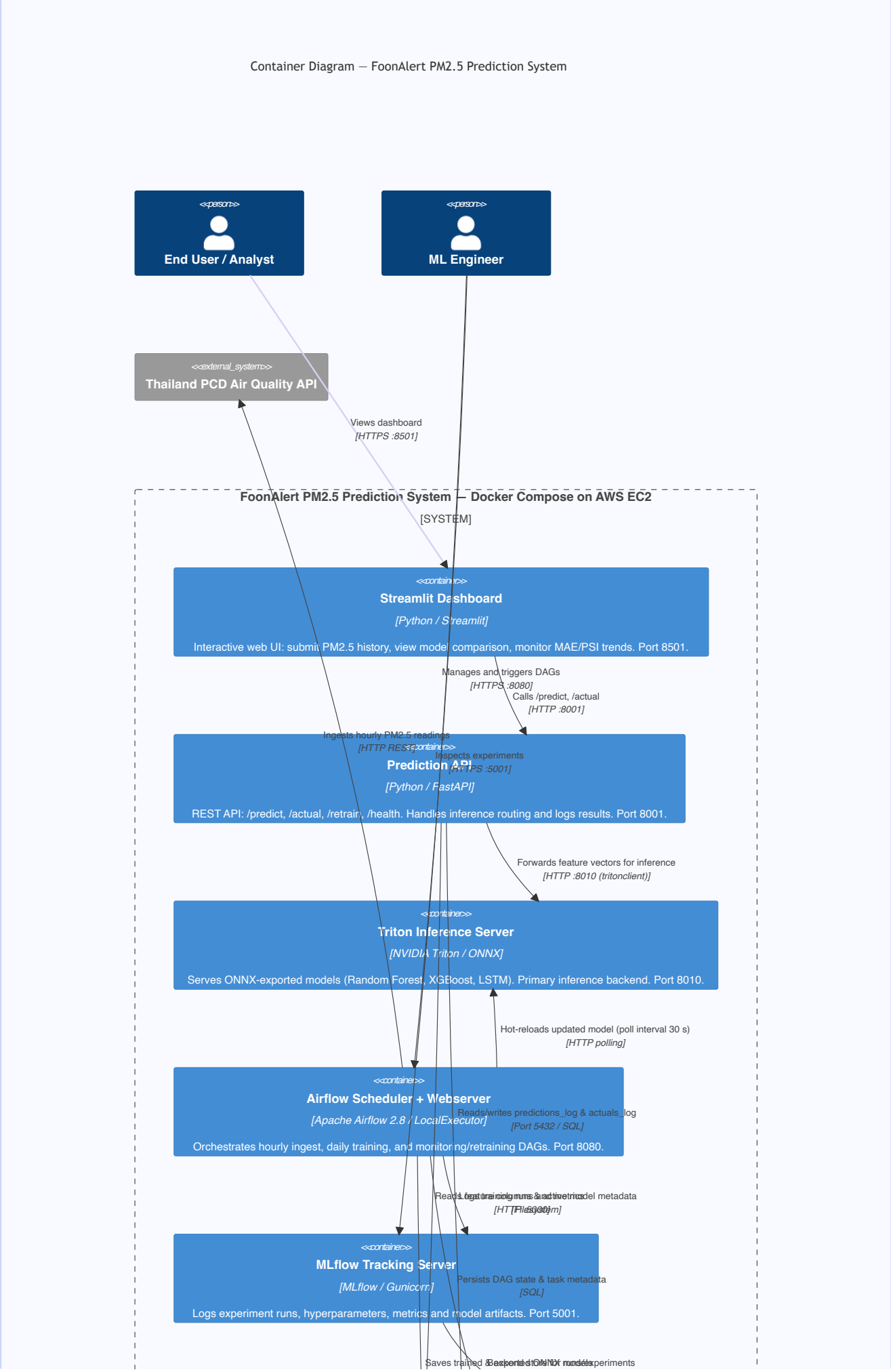
Design Decisions:

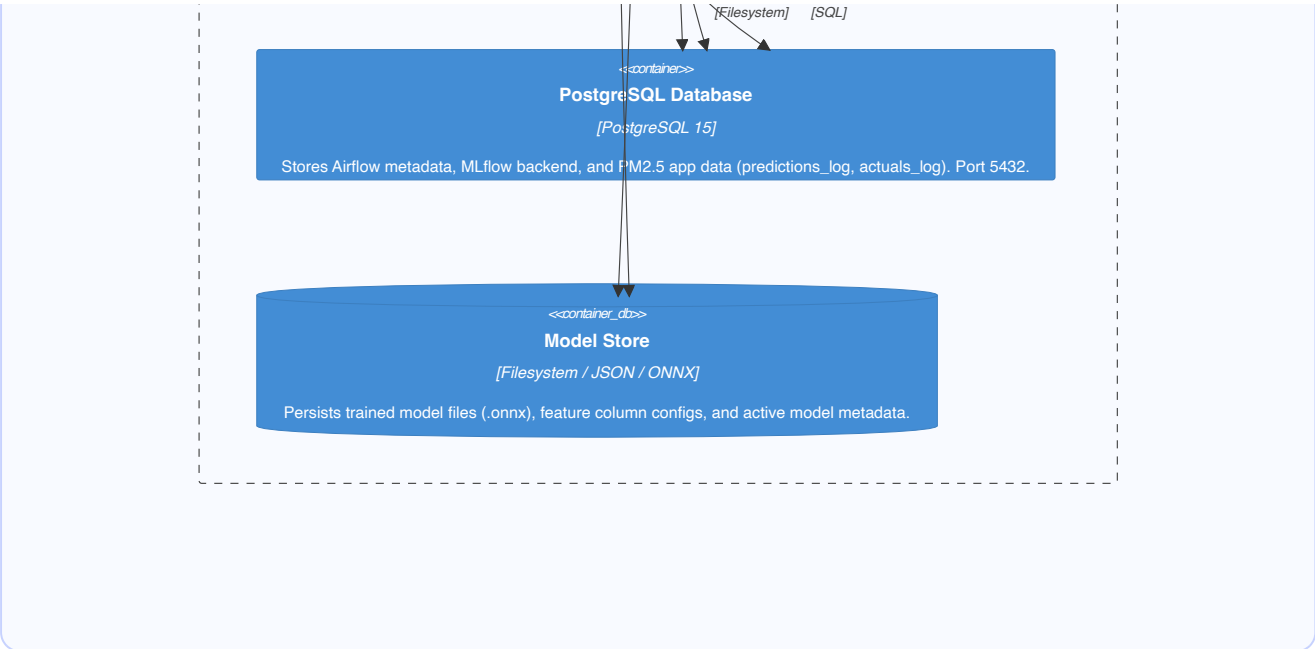
- The system is designed as a self-contained ML platform. MLflow is deployed inside the same Docker Compose stack rather than an external SaaS to avoid data egress costs and keep experiment history reproducible.

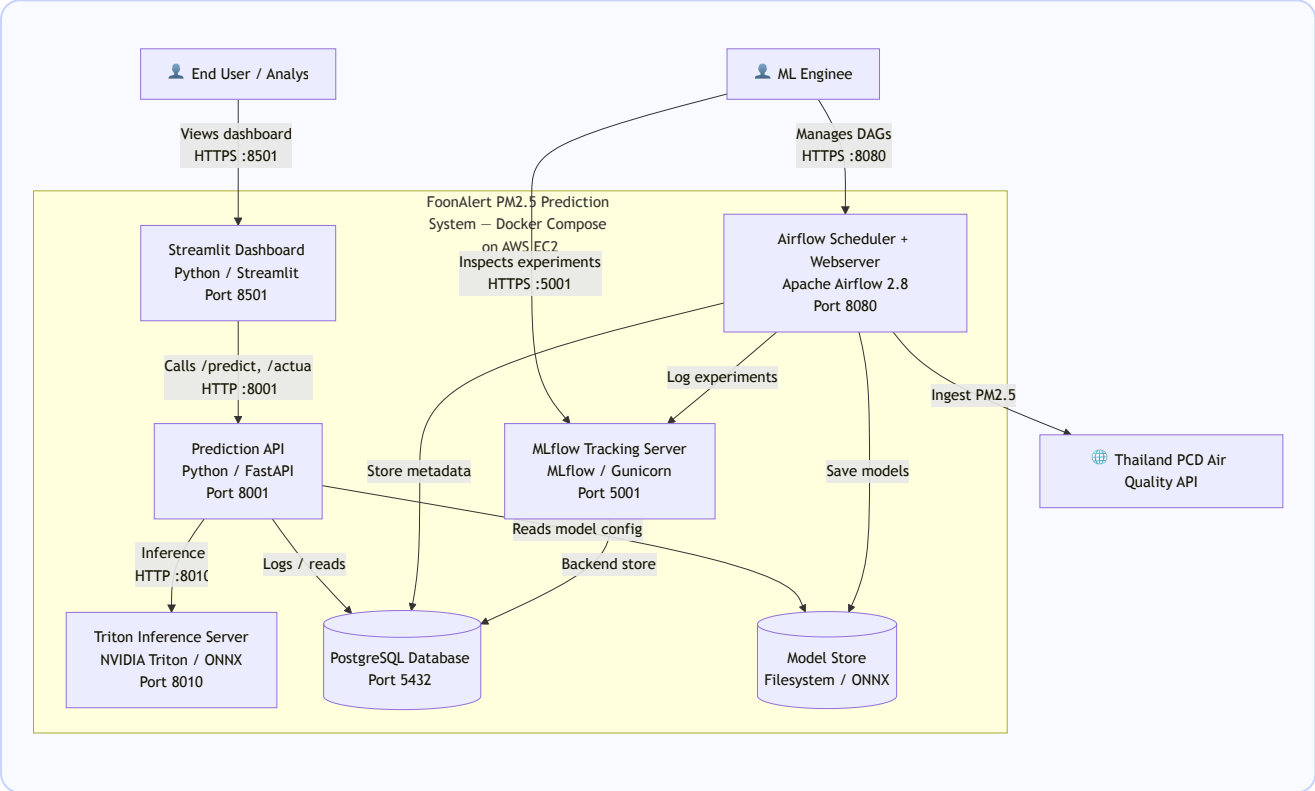
- Data ingestion is decoupled from prediction via Airflow DAGs, so the external PCD API outage does not block inference.

Level 2 — Container Diagram

> *Major system components and technologies.*


Container Diagram — FoonAlert PM2.5 Prediction System
<<person>>
End User / Analyst
<<person>>
ML Engineer
<<external_system>>
Thailand PCD Air Quality API
Views dashboard
[HTTPS :8501]
FoonAlert PM2.5 Prediction System — Docker Compose on AWS EC2
[SYSTEM]
<<container>>
Streamlit Dashboard
[Python / Streamlit]
Interactive web UI: submit PM2.5 history, view model comparison, monitor MAE/PSI trends. Port 8501.
Manages and triggers DAGs
[HTTPS :8080]
Calls /predict, /actual
[HTTP :8001]
Ingests hourly PM2.5 readings
[HTTP REST]
Inspects experiments
[HTTPS :5001]
<<container>>
Prediction API
[Python / FastAPI]
REST API: /predict, /actual, /retrain, /health. Handles inference routing and logs results. Port 8001.
Forwards feature vectors for inference
[HTTP :8010 (tritonclient)]
<<container>>
Triton Inference Server
[NVIDIA Triton / ONNX]
Serves ONNX-exported models (Random Forest, XGBoost, LSTM). Primary inference backend. Port 8010.
Hot-reloads updated model (poll interval 30 s)
[HTTP polling]
<<container>>
Airflow Scheduler + Webserver
[Apache Airflow 2.8 / LocalExecutor]
Orchestrates hourly ingest, daily training, and monitoring/retraining DAGs. Port 8080.
Reads/writes predictions_log & actuals_log
[Port 5432 / SQL]
[HTTP :5000]
<<container>>
MLflow Tracking Server
[MLflow / Gunicorn]
Logs experiment runs, hyperparameters, metrics and model artifacts. Port 5001.
Persists DAG state & task metadata
[SQL]
Saves trained ONNX models



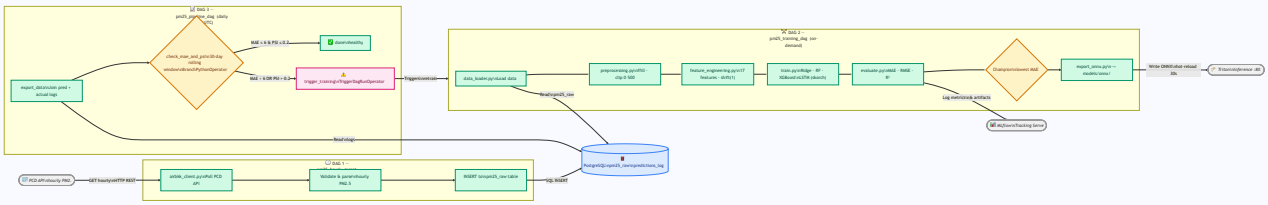


Design Decisions & Trade-offs:

| Decision | Rationale | |---|---| | Triton as primary inference backend | Handles ONNX models uniformly; supports GPU acceleration for LSTM without code changes. Falls back to local ONNX Runtime if Triton is unavailable. | | LocalExecutor (no Celery/k8s) | Simpler ops for a single-node deployment; sufficient for daily training cadence. | | Shared PostgreSQL | Reduces infrastructure cost; Airflow, MLflow, and app DB coexist in separate schemas. | | ONNX export for all models | Decouples training framework (sklearn, XGBoost, PyTorch) from the runtime; Triton can serve all formats via a single interface. |

Level 3 — Component Diagram (ML Pipeline Container)

> *Internal structure of the Airflow ML Pipeline.*



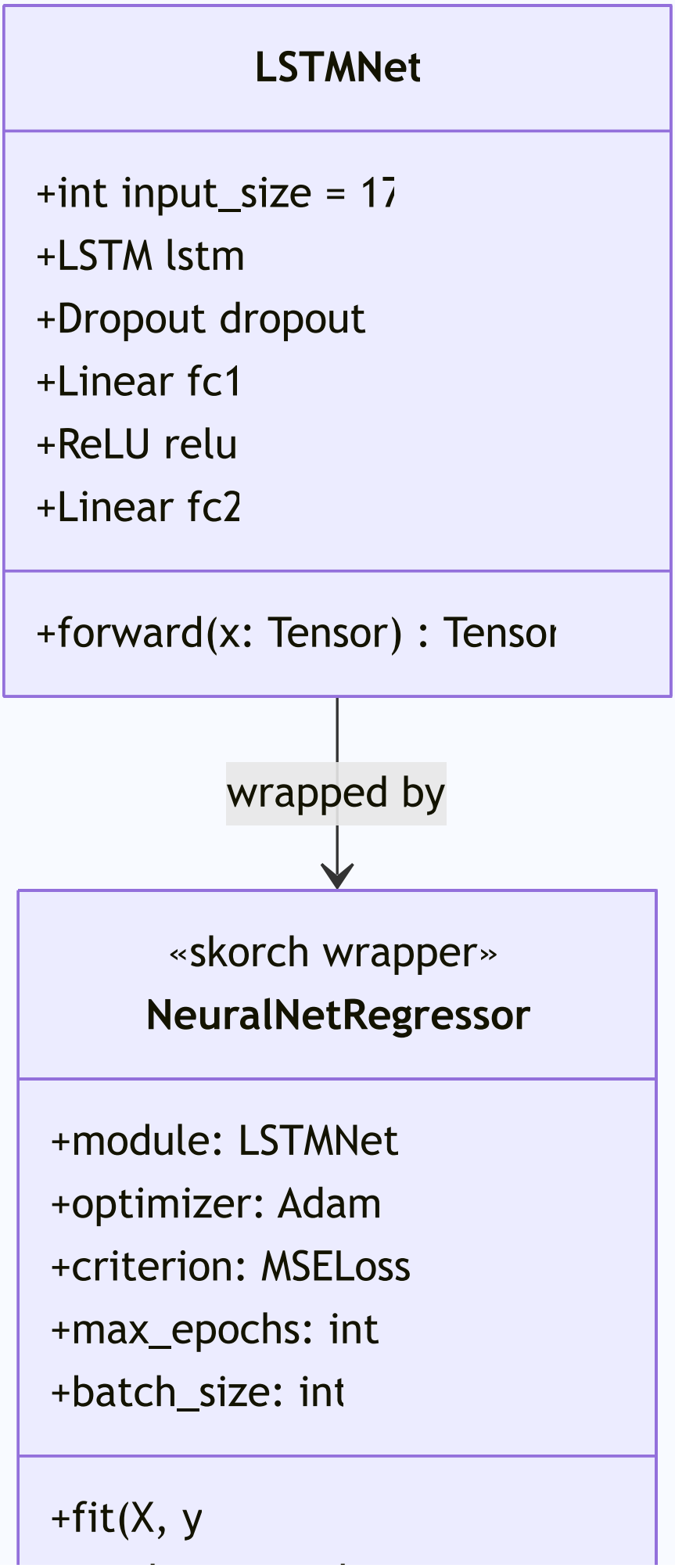
Design Decisions:

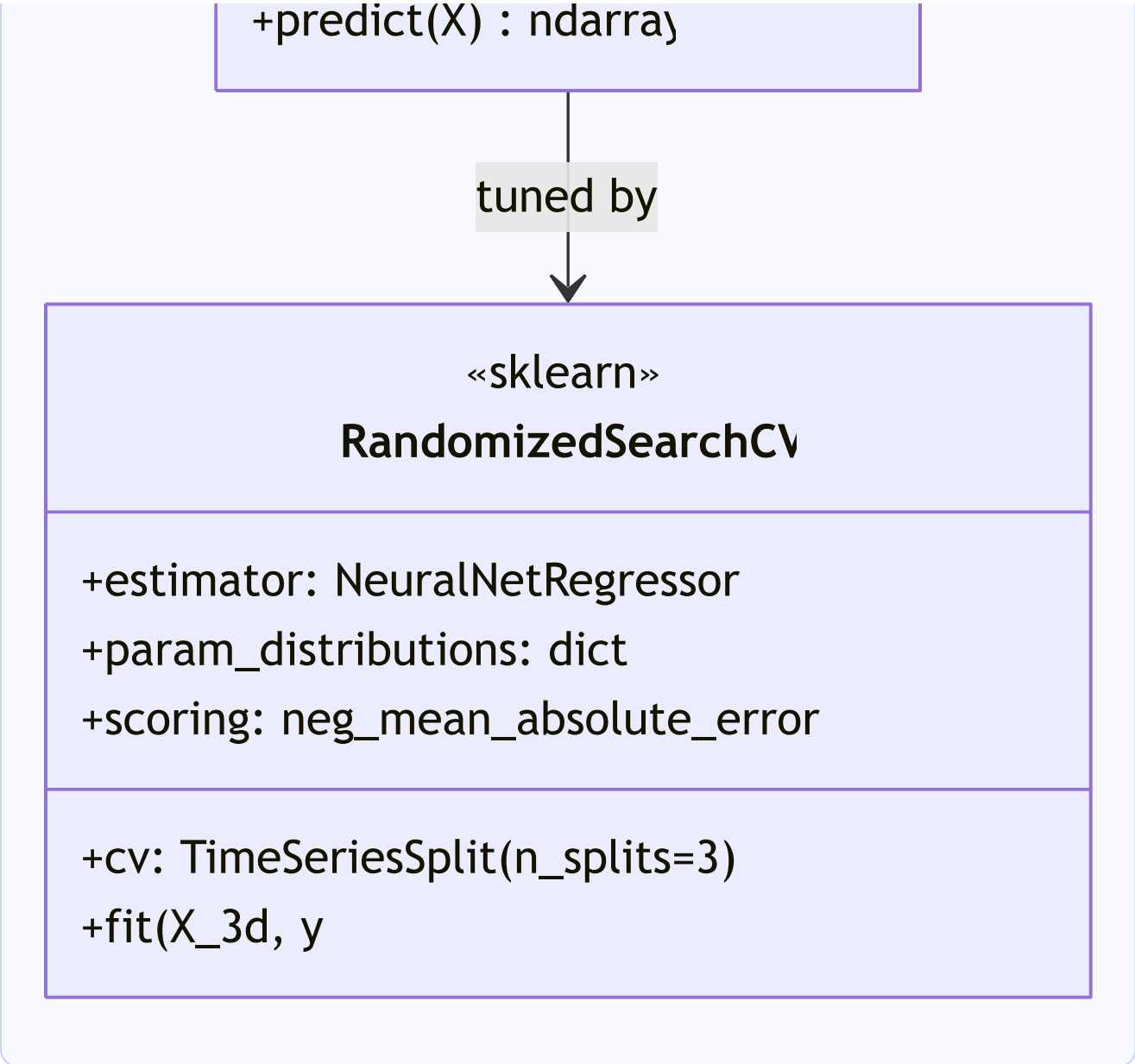
- **BranchPythonOperator** is used in `pm25_pipeline_dag` so Airflow skips downstream tasks cleanly when no retrain is needed — avoids false "failed" states.
- `shift(1)` applied to all rolling/lag features ensures no future data leaks into training.
- Champion model selection is metric-driven (lowest MAE on test split); ONNX export uses the same champion path for all algorithm families.

Level 4 — Code-Level Diagram (ML Component)

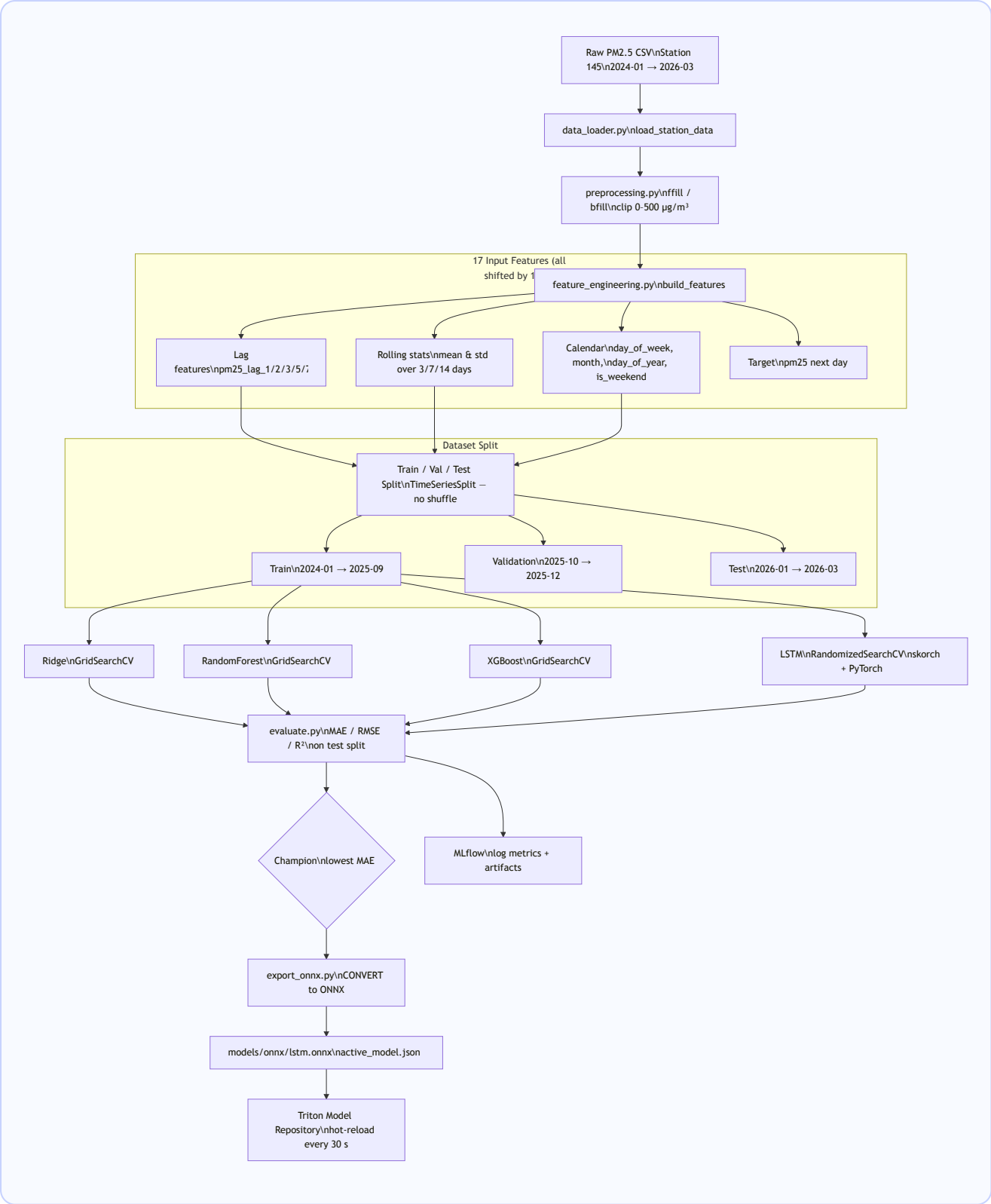
> *Key implementation structures inside the ML component.*

4a. LSTM Model Architecture (`src/lstm_model.py`)

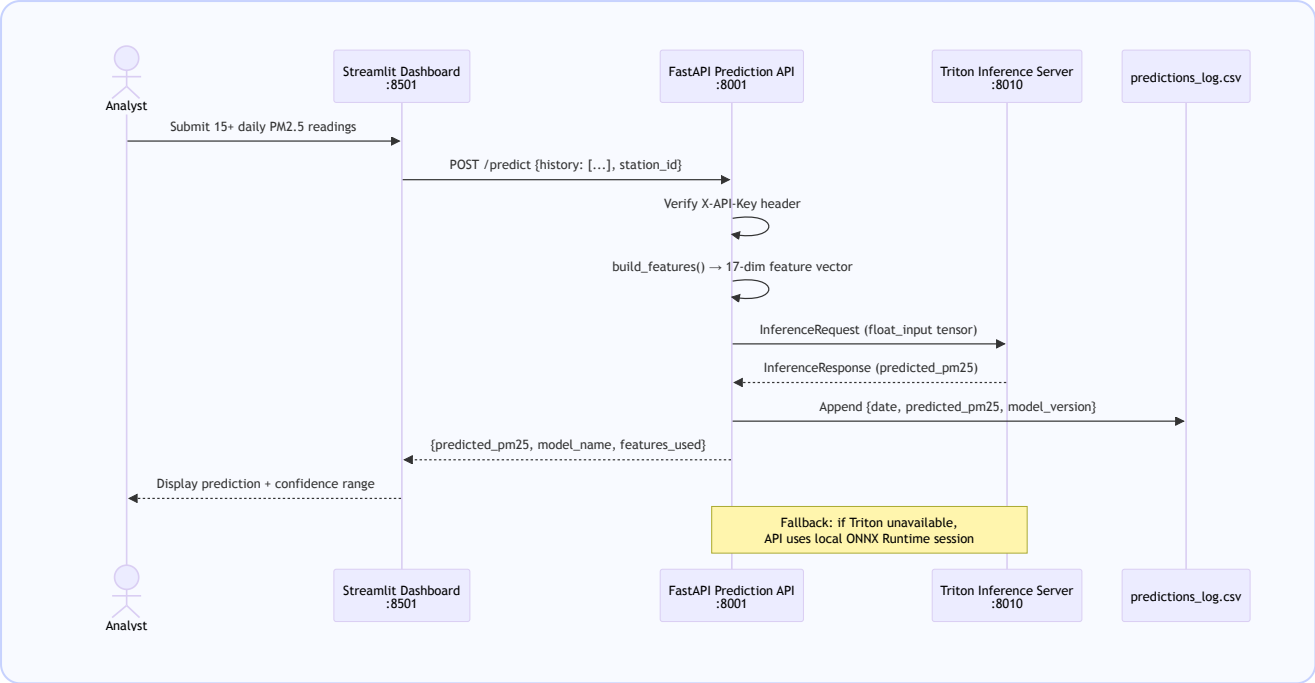

LSTMNet
+int input_size = 17
+LSTM lstm
+Dropout dropout
+Linear fc1
+ReLU relu
+Linear fc2
+forward(x: Tensor) : Tensor
wrapped by
«skorch wrapper»
NeuralNetRegressor
+module: LSTMNet
+optimizer: Adam
+criterion: MSELoss
+max_epochs: int
+batch_size: int
+fit(X, y



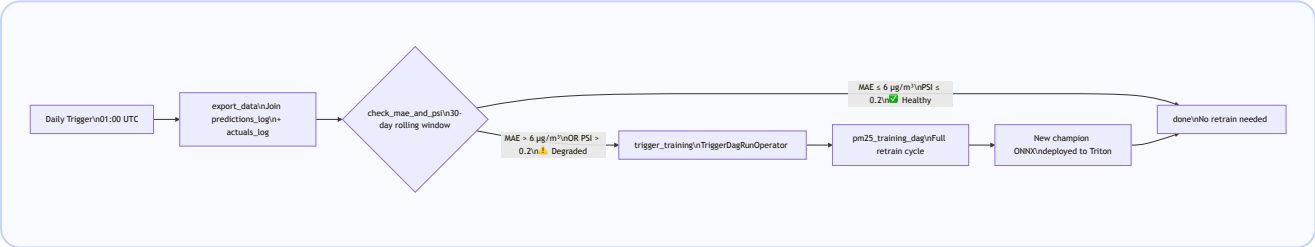
4b. Training Pipeline Data Flow (`src/train.py` + `src/feature_engineering.py`)



4c. Online Inference Flow (`src/api.py`)



4d. Monitoring & Auto-Retrain Flow (`dags/pm25_pipeline_dag.py`)



Summary of Architecture Decisions

| Concern | Decision | Trade-off | |---|---|---| | **Inference latency** | Triton serves ONNX; FastAPI is stateless | Adds one network hop vs in-process inference, but enables horizontal scaling | | **Model flexibility** | All models exported to ONNX | Loses PyTorch dynamic graph features; static input shape required | | **Data leakage prevention** | `shift(1)` on all lag/rolling features | Slightly reduces short-term signal but is production-safe | | **Retraining trigger** | Dual signal: MAE + PSI | PSI catches distribution shift before MAE degrades; may trigger unnecessary retrains | | **Data split** | Strict temporal (no shuffle) | Realistic evaluation; smaller effective training set vs random split | | **Auth** | API key header (`X-API-Key`) | Simple to implement; suitable for internal / dashboard-only calls |